

Le backend, ligne par ligne

RockYou Password Checker — API FastAPI de vérification de compromission de mots de passe (k-anonymat) et de conformité ANSSI. Chaque instruction du code source est isolée et commentée individuellement.

Projet : rockyou-checker / backend

Généré le : 15 août 2026

Couverture : tous les fichiers Python du backend, la configuration (Docker, .env, pyproject), et les tests — rien n'est laissé de côté.

0. Avertissement & méthodologie

Je n'ai pas créé ce backend. Il existait déjà, complet, dans votre projet avant cette conversation. Mon rôle est de le **lire** et de l'**expliquer**, pas de l'avoir écrit. Je n'ai donc aucune commande de création à vous restituer (pas de scaffolding, pas de `pip install` initial) — en inventer serait inexact.

Les seules commandes exécutées pour préparer ce document sont des commandes de lecture, sans aucune modification :

```
find ./backend -type f -not -path '*/__pycache__/*' | sort
```

puis l'ouverture individuelle, en lecture seule, de chaque fichier source du dossier `backend/`.

Sur la fraîcheur des versions de bibliothèques (section 4) : la vérification en direct sur PyPI n'a pas pu aboutir dans cette session (appel réseau interrompu). Pour un état exact : `pip list --outdated` et `pip-audit -r requirements.txt`.

1. Vue d'ensemble

Une API FastAPI qui fait deux choses : vérifier si un mot de passe a fuité (base RockYou) et évaluer sa conformité ANSSI — sans jamais recevoir le mot de passe en clair.

```
Client (frontend)
| hash SHA-1 du mot de passe calcule LOCALEMENT
| seuls les 5 premiers caracteres du hash sont envoyes (k-anonymat)
v
[main.py]          -> cree l'app FastAPI, branche les middlewares
                   (CORS, en-tetes de securite, rate limit)
v
[api/routes.py]    -> endpoints /health et /breach-check
v
[api/schemas.py]  -> validation requete/reponse (Pydantic)
v
[db/db.py]         -> interroge PostgreSQL par prefixe de hash
v
[core/config.py] + [core/rate_limit.py] -> configuration et anti-abus
```

Sommaire des fichiers couverts

- 2.1 app/core/config.py — configuration
- 2.2 app/core/rate_limit.py — limitation de débit
- 2.3 app/db/db.py — accès PostgreSQL
- 2.4 app/api/schemas.py — validation des données
- 2.5 app/api/routes.py — endpoints
- 2.6 app/main.py — point d'entrée
- 2.7 app/rules/anssi.py + anssi_rules.json — scoring ANSSI
- 2.8 database/init.sql — schéma PostgreSQL
- 2.9 .env.example — variables d'environnement
- 2.10 Dockerfile — image de production
- 2.11 pyproject.toml — configuration des outils
- 2.12 requirements-dev.txt — dépendances de développement
- 2.13 tests/test_anssi.py — tests du scoring ANSSI
- 2.14 tests/test_anssi_rules_sync.py — test de cohérence backend/frontend

2. Fichiers, ligne par ligne

2.1 core/config.py

backend/app/core/config.py – configuration de l'application

```
from __future__ import annotations
```

Import spécial (« future ») qui change la façon dont Python évalue les annotations de type dans *tout le fichier* : elles deviennent des chaînes de caractères non évaluées à la définition, mais seulement à la demande. Concrètement, ça permet d'écrire `str | None` (syntaxe moderne) même sous des versions de Python où ce n'était pas nativement supporté, sans risque d'erreur au chargement du module.

```
from functools import lru_cache
```

`lru_cache` est un décorateur de la bibliothèque standard qui mémorise le résultat d'une fonction : au premier appel, il exécute la fonction et garde le résultat en cache ; à chaque appel suivant avec les mêmes arguments, il renvoie directement la valeur en cache sans ré-exécuter le code. « LRU » = Least Recently Used, la politique d'éviction du cache si une taille max était fixée (ici non pertinent car la fonction n'a aucun argument).

```
from pydantic_settings import BaseSettings, SettingsConfigDict
```

`BaseSettings` est une classe de base de la bibliothèque `pydantic-settings` : contrairement à un `BaseModel` Pydantic classique (qui valide des données déjà en main), `BaseSettings` va lui-même *chercher* ses valeurs dans l'environnement du processus (variables d'env, fichier `.env`).

`SettingsConfigDict` est le type utilisé pour configurer ce comportement.

```
class Settings(BaseSettings):
```

Déclare la classe `Settings`, qui hérite de `BaseSettings`. Chaque attribut de classe annoté ci-dessous deviendra automatiquement une source de configuration lue depuis l'environnement.

```
model_config = SettingsConfigDict(env_file=".env", extra="ignore")
```

- `env_file=".env"` : si un fichier nommé `.env` existe dans le répertoire courant au démarrage, ses lignes `CLE=valeur` sont chargées comme si elles étaient des variables d'environnement.
- `extra="ignore"` : si le fichier `.env` ou l'environnement contient des clés qui ne correspondent à aucun attribut déclaré plus bas, elles sont silencieusement ignorées plutôt que de lever une erreur de validation.

```
database_url: str = "postgres://rockyou:rockyou@postgres:5432/rockyou"
```

Lu depuis la variable d'environnement `DATABASE_URL` (Pydantic convertit automatiquement le nom en *snake_case* vers la variante majuscule). La valeur par défaut pointe vers le service `postgres` du `docker-compose.yml`, utile en développement conteneurisé sans avoir à définir explicitement la variable.

```
api_cors_origins: str = "http://localhost:5173"
```

Chaîne brute (potentiellement une liste séparée par des virgules) des origines autorisées à appeler l'API depuis un navigateur. `5173` est le port par défaut du serveur de développement Vite utilisé par le frontend.

```
rate_limit_per_minute: int = 30
```

En réalité, ce réglage est bien utilisé : la route de `api/routes.py` applique `@limiter.limit(lambda: f"{get_settings().rate_limit_per_minute}/minute")`, qui relit cette valeur à chaque requête plutôt que de la figer au démarrage — voir 2.5.

```
db_pool_min_size: int = 2
db_pool_max_size: int = 10
```

Bornes du pool de connexions PostgreSQL ouvert dans `db/db.py` : au minimum 2 connexions maintenues ouvertes en permanence, au maximum 10 en cas de charge simultanée.

```
environment: str = "development"
```

Marqueur d'environnement d'exécution (« development », « production »...). Déclaré mais non lu ailleurs dans le code actuel — un point d'extension prévu plus qu'un mécanisme actif.

```
@property
def cors_origins_list(self) -> list[str]:
    return [o.strip() for o in self.api_cors_origins.split(",") if o.strip()]
```

`@property` transforme une méthode en attribut calculé : on l'utilise comme `settings.cors_origins_list`, sans parenthèses.

La ligne de retour est une *list comprehension* qui s'exécute en 3 temps :

- `self.api_cors_origins.split(",")` : découpe la chaîne `"http://a.com,http://b.com"` en `["http://a.com", "http://b.com"]` ;
- `for o in ...` : itère sur chaque morceau ;
- `o.strip()` : retire les espaces superflus autour de chaque origine (utile si l'utilisateur écrit `"a.com, b.com"` avec un espace après la virgule) ;
- `if o.strip()` : filtre les entrées vides (ex. une virgule finale accidentelle) avant qu'elles n'entrent dans la liste finale.

```
@lru_cache
def get_settings() -> Settings:
    return Settings()
```

Fonction d'accès unique à la configuration. Le décorateur `@lru_cache` vu plus haut garantit que `Settings()` — qui relit potentiellement le fichier `.env` et l'environnement — n'est instancié qu'**une seule fois** pour tout le processus, puis renvoyé tel quel à chaque appel ultérieur de `get_settings()`. C'est le point d'entrée utilisé par tous les autres modules (`db.py`, `main.py`).

2.2 core/rate_limit.py

[backend/app/core/rate_limit.py – anti-abus](#)

```
from __future__ import annotations
```

Même rôle que dans `config.py` : résolution différée des annotations de type.

```
from slowapi import Limiter
from slowapi.util import get_remote_address
```

`Limiter` est la classe centrale de la bibliothèque `slowapi`, qui implémente le rate limiting pour Starlette/FastAPI. `get_remote_address` est une fonction utilitaire fournie par la bibliothèque qui extrait l'adresse IP du client à partir de l'objet `Request`.

```
limiter = Limiter(key_func=get_remote_address)
```

Instancie **un seul** limiteur pour toute l'application. `key_func=get_remote_address` indique comment regrouper les requêtes pour le comptage : ici, par adresse IP — deux requêtes de la même IP partagent le même quota, deux IP différentes ont chacune leur propre compteur. Par défaut, `slowapi` stocke ces compteurs **en mémoire du processus** (pas de Redis externe configuré ici), ce qui signifie que le quota est réinitialisé si le serveur redémarre, et n'est pas partagé entre plusieurs instances du backend si on le fait tourner à l'échelle (limite acceptable pour ce projet, à surveiller si un jour il tourne en plusieurs répliques).

2.3 db/db.py

[backend/app/db/db.py – accès PostgreSQL](#)

```
"""
Accès PostgreSQL : cycle de vie du pool de connexions et requête de
recherche par préfixe de hash (modèle de confidentialité k-anonymat,
voir README).
"""
```

Docstring de module : la triple-guillemet en tête de fichier documente l'intention du module entier, visible via `help(module)` ou par les outils de documentation automatique.

```
from dataclasses import dataclass
import asyncpg
from app.core.config import get_settings
```

`dataclass` : décorateur qui génère automatiquement `__init__`, `__repr__` et `__eq__` pour une classe qui ne sert qu'à porter des données. `asyncpg` : driver PostgreSQL asynchrone (pas d'ORM, requêtes SQL directes, très performant). `get_settings` : accès à la configuration vue en 2.1.

```
@dataclass(frozen=True)
class SuffixMatch:
    suffix: str
    occurrence_count: int
```

`frozen=True` rend les instances **immuables** : toute tentative de modifier `suffix` ou `occurrence_count` après création lève une exception. C'est une structure de transport interne, entre la requête SQL et la conversion en réponse HTTP — `suffix` est la partie du hash après les 5 premiers caractères, `occurrence_count` le nombre de fois que ce mot de passe apparaît dans le corpus RockYou.

```
class Database:
    """Encapsule le cycle de vie du pool de connexions asyncpg."""

    def __init__(self) -> None:
        self._pool: asyncpg.Pool | None = None
```

Le préfixe `_` sur `_pool` est une convention Python signalant un attribut « privé » (pas d'application stricte par le langage, juste un signal aux lecteurs du code). Il vaut `None` tant qu'aucune connexion n'a été établie — c'est-à-dire entre l'import du module et l'appel à `connect()` au démarrage de l'application.

```
async def connect(self) -> None:
    settings = get_settings()
    self._pool = await asyncpg.create_pool(
        dsn=settings.database_url,
        min_size=settings.db_pool_min_size,
        max_size=settings.db_pool_max_size,
        command_timeout=5,
    )
```

`async def` : fonction asynchrone, doit être appelée avec `await`. `asyncpg.create_pool(...)` est elle-même une coroutine qui établit les connexions initiales et retourne un objet `Pool`.

- `dsn=settings.database_url` : la chaîne de connexion complète (utilisateur, mot de passe, hôte, port, nom de base) ;
- `min_size` / `max_size` : bornes du pool vues en 2.1 ;
- `command_timeout=5` : toute requête SQL individuelle qui dépasse 5 secondes est annulée et lève une exception, plutôt que de bloquer indéfiniment une connexion (et donc, à terme, tout le pool).

```

async def disconnect(self) -> None:
    if self._pool is not None:
        await self._pool.close()
        self._pool = None

```

La vérification `if self._pool is not None` rend la méthode sûre à appeler même si `connect()` n'a jamais réussi (évite un `AttributeError` sur `None`). `pool.close()` attend la fin des requêtes en cours puis ferme proprement toutes les connexions physiques. Remettre `self._pool` à `None` permet de ré-appeler `connect()` plus tard si nécessaire (utile notamment dans les tests).

```

@property
def pool(self) -> asyncpg.Pool:
    if self._pool is None:
        raise RuntimeError("Le pool de base de données n'est pas initialisé.")
    return self._pool

```

Accesseur qui transforme une erreur silencieuse potentielle (`None.acquire()` → `AttributeError` cryptique) en une erreur explicite et compréhensible, si jamais du code tentait d'utiliser la base avant l'appel à `connect()` dans le `lifespan` de `main.py`.

```

database = Database()

```

Instanciation au niveau du module : cet objet est créé **une seule fois**, à l'import du module, et c'est cette instance unique (un singleton de fait) qui est importée et partagée par `main.py` (pour `connect` / `disconnect`) et par les fonctions `find_by_prefix` / `health_check` ci-dessous.

```

_SELECT_BY_PREFIX = """
    SELECT hash_suffix, occurrence_count
    FROM breached_passwords
    WHERE hash_prefix = $1
    """

```

Constante de module (préfixe `_` = privée au fichier) contenant la requête SQL, définie une seule fois plutôt qu'à chaque appel de fonction.

- `SELECT hash_suffix, occurrence_count` : ne récupère que les deux colonnes nécessaires, pas `SELECT *` ;
- `FROM breached_passwords` : la table créée par `init.sql` (voir 2.8) ;
- `WHERE hash_prefix = $1` : `$1` est un **paramètre positionnel** au sens du protocole PostgreSQL — sa valeur sera fournie séparément par `asyncpg`, jamais concaténée dans la chaîne SQL. C'est ce qui rend la requête totalement immunisée contre l'injection SQL.

```

async def find_by_prefix(prefix: str) -> list[SuffixMatch]:
    async with database.pool.acquire() as connection:
        rows = await connection.fetch(_SELECT_BY_PREFIX, prefix)
        return [SuffixMatch(row["hash_suffix"], row["occurrence_count"]) for row in rows]

```

- `database.pool.acquire()` : emprunte une connexion physique au pool. C'est un *context manager asynchrone* (`async with`) : la connexion est automatiquement rendue au pool en sortie du bloc, même si une exception survient à l'intérieur ;
- `connection.fetch(_SELECT_BY_PREFIX, prefix)` : exécute la requête en substituant `prefix` à `$1`, et retourne **toutes** les lignes correspondantes sous forme d'objets `Record` (accessibles comme des dictionnaires, ex. `row["hash_suffix"]`) ;
- la ligne de retour est une *list comprehension* qui convertit chaque `Record` brut en un objet `SuffixMatch` typé, défini plus haut dans ce même fichier.

Point de conception central : le paramètre d'entrée est le **préfixe** (5 caractères) — jamais le hash complet, jamais le mot de passe. C'est le cœur du modèle k-anonymat de tout le projet.

```
async def health_check() -> bool:
    async with database.pool.acquire() as connection:
        result = await connection.fetchval("SELECT 1")
    return result == 1
```

`fetchval` (plutôt que `fetch`) retourne directement la valeur scalaire de la première colonne de la première ligne, sans passer par une liste de `Record` — adapté ici puisqu'on attend une seule valeur (`1`). Utilisé par l'endpoint `/health` de `routes.py` pour confirmer que la base de données répond réellement, pas seulement que le pool existe.

2.4 api/schemas.py

backend/app/api/schemas.py – validation des données

```
from pydantic import BaseModel, Field, field_validator
```

`BaseModel` : classe de base pour tout schéma Pydantic (validation + sérialisation). `Field` : permet d'ajouter des contraintes et métadonnées à un champ (longueur, exemples...). `field_validator` : décorateur pour écrire une règle de validation personnalisée sur un champ précis.

```
class BreachCheckRequest(BaseModel):
```

Modèle représentant le corps JSON attendu par l'endpoint `POST /breach-check`. FastAPI l'utilise pour parser et valider automatiquement chaque requête entrante avant d'exécuter le code de la route.

```
hash_prefix: str = Field(..., min_length=5, max_length=5, examples=["5BAA6"])
```

- `str` : type attendu ;
- `Field(...)` : le premier argument positionnel `...` (l'objet `Ellipsis` de Python) signifie « champ obligatoire, pas de valeur par défaut » ;
- `min_length=5, max_length=5` : la chaîne doit faire **exactement** 5 caractères — toute requête avec un préfixe plus court ou plus long est rejetée automatiquement par Pydantic avec une

réponse HTTP 422, avant même d'exécuter la moindre ligne de `routes.py` ;

- `examples=["5BAA6"]` : n'a aucun effet sur la validation — sert uniquement à enrichir la documentation Swagger générée sur `/api/docs`.

```
@field_validator("hash_prefix")
@classmethod
def validate_hex(cls, value: str) -> str:
    value = value.upper()
    if not all(c in "0123456789ABCDEF" for c in value):
        raise ValueError("hash_prefix doit être hexadécimal (0-9, A-F).")
    return value
```

- `@field_validator("hash_prefix")` : enregistre cette fonction comme validateur supplémentaire pour le champ `hash_prefix`, exécuté *après* la validation de longueur ci-dessus ;
- `@classmethod` : requis par Pydantic v2 pour ce type de validateur — `cls` reçoit la classe elle-même, pas une instance ;
- `value = value.upper()` : normalise la casse avant comparaison et avant stockage, pour que `"5baa6"` et `"5BAA6"` soient traités de façon identique ;
- `all(c in "0123456789ABCDEF" for c in value)` : vérifie caractère par caractère que chacun appartient bien à l'alphabet hexadécimal ; `all(...)` retourne `False` dès le premier caractère invalide ;
- `raise ValueError(...)` : Pydantic intercepte automatiquement cette exception et la transforme en réponse HTTP 422 avec un message d'erreur structuré ;
- `return value` : un validateur Pydantic doit toujours retourner la valeur (éventuellement transformée) qui sera effectivement stockée sur l'instance.

```
class SuffixMatchResponse(BaseModel):
    hash_suffix: str
    occurrence_count: int
```

Représente une entrée individuelle du tableau de résultats renvoyé au client : un suffixe possible et le nombre de fois qu'il paraît dans le corpus RockYou.

```
class BreachCheckResponse(BaseModel):
    hash_prefix: str
    matches: list[SuffixMatchResponse]
```

Modèle de la réponse complète : le préfixe interrogé (renvoyé pour confirmation) et la liste — potentiellement vide si le préfixe ne correspond à aucune fuite connue — des correspondances trouvées.

```
class HealthResponse(BaseModel):
    status: str
    database: bool
```

Modèle de réponse du endpoint `/health` : un statut textuel (`"ok"` ou `"degraded"`) et un booléen indiquant si la base de données a répondu.

2.5 api/routes.py

[backend/app/api/routes.py](#) – les endpoints

Ce fichier n'utilise volontairement **pas** `from __future__ import annotations` : combiné au décorateur `@limiter.limit(...)` de `slowapi`, cet import empêcherait Pydantic de résoudre le type `BreachCheckRequest` au démarrage (`PydanticUndefinedAnnotation`) et ferait planter l'application — c'est documenté explicitement en commentaire en tête du fichier source.

```
from fastapi import APIRouter, Request

from app.api.schemas import (
    BreachCheckRequest,
    BreachCheckResponse,
    HealthResponse,
    SuffixMatchResponse,
)
from app.core.rate_limit import limiter
from app.db import db

router = APIRouter()
```

`APIRouter` : conteneur de routes indépendant de l'application principale, qui sera « monté » dessus dans `main.py` avec un préfixe commun. Chaque endpoint ci-dessous est déclaré via un décorateur sur `router` (`@router.get` , `@router.post`) plutôt que directement sur `app` .

```
@router.get("/health", response_model=HealthResponse, tags=["monitoring"])
async def health() -> HealthResponse:
```

`@router.get("/health", ...)` déclare un endpoint `GET` . `response_model=HealthResponse` : FastAPI validera et sérialisera automatiquement la valeur de retour selon ce schéma, et l'inclura dans la documentation OpenAPI. `tags=["monitoring"]` : regroupe visuellement cet endpoint sous la catégorie « monitoring » dans `/api/docs` .

```
try:
    db_ok = await db.health_check()
except Exception:
    db_ok = False
return HealthResponse(status="ok" if db_ok else "degraded", database=db_ok)
```

Capture **toute** exception (`except Exception` , volontairement large ici) pouvant survenir si la base est injoignable — pool non initialisé, timeout réseau, etc. — pour que l'endpoint de supervision réponde toujours `200 OK` avec un statut `"degraded"` plutôt que de renvoyer une erreur 500 qui casserait un système de monitoring externe.

```
@router.post("/breach-check", response_model=BreachCheckResponse, tags=["breach"])
@limiter.limit(lambda: f"{get_settings().rate_limit_per_minute}/minute")
async def breach_check(
    request: Request, # requis par slowapi
    payload: BreachCheckRequest,
) -> BreachCheckResponse:
```

- `@router.post(...)` : endpoint `POST`, avec le même mécanisme de `response_model` / `tags` que `/health` ;
- `@limiter.limit(lambda: ...)` : relit `rate_limit_per_minute` depuis la configuration à chaque requête, au lieu d'une valeur figée (30/minute par défaut) ;
- `request: Request` : slowapi a besoin d'accéder à l'objet requête brut (pour en extraire l'IP via `get_remote_address`) — ce paramètre doit être présent même s'il n'est jamais utilisé explicitement dans le corps de la fonction ;
- `payload: BreachCheckRequest` : FastAPI lit le corps JSON de la requête, le valide via le schéma vu en 2.4, et injecte l'instance validée directement ici — aucune étape manuelle de parsing.

```
matches = await db.find_by_prefix(payload.hash_prefix)
return BreachCheckResponse(
    hash_prefix=payload.hash_prefix,
    matches=[
        SuffixMatchResponse(hash_suffix=m.suffix, occurrence_count=m.occurrence_count)
        for m in matches
    ],
)
```

`db.find_by_prefix(...)` retourne une liste d'objets internes `SuffixMatch` (2.3). La liste les convertit un par un en `SuffixMatchResponse`, le modèle de *réponse publique* — une séparation volontaire entre la représentation interne (base de données) et le contrat d'API exposé, même si les deux se ressemblent aujourd'hui. Le client reçoit tous les suffixes correspondant au préfixe envoyé et compare **localement** le reste de son propre hash : le serveur ne sait jamais quel mot de passe précis était recherché.

2.6 main.py

[backend/app/main.py](#) — point d'entrée

```

from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from slowapi import _rate_limit_exceeded_handler
from slowapi.errors import RateLimitExceeded
from slowapi.middleware import SlowAPIMiddleware
from starlette.middleware.base import BaseHTTPMiddleware
from starlette.responses import Response

from app.api.routes import router
from app.core.config import get_settings
from app.core.rate_limit import limiter
from app.db.db import database

```

`asynccontextmanager` : décorateur pour écrire un gestionnaire de contexte asynchrone avec la syntaxe simple `yield` plutôt qu'une classe complète avec `__aenter__` / `__aexit__`.

`CORSMiddleware` : middleware CORS fourni par FastAPI. Les trois imports `slowapi` apportent le handler d'erreur, l'exception de dépassement de quota, et le middleware de comptage.

`BaseHTTPMiddleware` / `Response` (Starlette, la couche ASGI sous-jacente à FastAPI) servent à écrire le middleware de sécurité maison ci-dessous.

```

class SecurityHeadersMiddleware(BaseHTTPMiddleware):
    """Ajoute des en-têtes de sécurité HTTP de base à chaque réponse."""

    async def dispatch(self, request: Request, call_next) -> Response:
        response = await call_next(request)

```

Hériter de `BaseHTTPMiddleware` et redéfinir `dispatch` est le patron standard pour écrire un middleware Starlette. `call_next(request)` transmet la requête à la suite de la chaîne de traitement (autres middlewares, puis la route elle-même) et retourne la `Response` qui en résulte — c'est seulement **après** cet appel qu'on peut modifier les en-têtes de la réponse sortante.

```
response.headers["X-Content-Type-Options"] = "nosniff"
```

Empêche le navigateur de « deviner » (sniffer) un type de contenu différent de celui déclaré par `Content-Type` — protection contre certaines attaques XSS où un fichier serait interprété comme du HTML/JS exécutable alors qu'il ne le devrait pas.

```
response.headers["X-Frame-Options"] = "DENY"
```

Interdit totalement l'affichage de la page dans une `<iframe>`, quel que soit le site qui l'intègre — protection anti-clickjacking (un attaquant ne peut pas superposer une iframe invisible du site par-dessus un autre pour piéger des clics).

```
response.headers["Referrer-Policy"] = "no-referrer"
```

Quand un lien de la page est cliqué vers un site externe, aucune information d'URL d'origine (« referrer ») n'est transmise au site de destination.

```
response.headers["Cache-Control"] = "no-store"
```

Interdit toute mise en cache de la réponse par le navigateur, un proxy intermédiaire ou une CDN — pertinent puisque les réponses de cette API sont liées, même indirectement, à des mots de passe.

```
return response
```

Renvoie la réponse modifiée pour qu'elle poursuive sa route vers le client.

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    await database.connect()
    yield
    await database.disconnect()
```

Tout le code avant `yield` s'exécute une seule fois, **au démarrage** de l'application (ouvre le pool de connexions vu en 2.3). L'exécution reste suspendue sur le `yield` pendant toute la durée de vie du serveur, le temps de traiter les requêtes entrantes. Tout le code après `yield` s'exécute une seule fois, **à l'arrêt** de l'application (ferme proprement le pool) — y compris en cas d'arrêt provoqué par un signal système (Ctrl+C, arrêt du conteneur).

```
def create_app() -> FastAPI:
    settings = get_settings()
```

Récupère l'instance unique de configuration (2.1) avant de construire l'application, pour pouvoir l'utiliser dans les réglages CORS plus bas.

```
app = FastAPI(
    title="RockYou Password Checker API",
    description(...),
    version="1.0.0",
    lifespan=lifespan,
    docs_url="/api/docs",
    openapi_url="/api/openapi.json",
)
```

`title` / `description` / `version` : métadonnées purement informatives, affichées dans la documentation générée. `lifespan=lifespan` branche le gestionnaire de cycle de vie défini juste au-dessus. `docs_url` / `openapi_url` déplacent la documentation Swagger et le schéma OpenAPI brut sous le préfixe `/api/...`, cohérent avec le reste des routes de l'application plutôt que les emplacements par défaut de FastAPI (`/docs`, `/openapi.json`).

```

app.state.limiter = limiter
app.add_exception_handler(RateLimitExceeded, _rate_limit_exceeded_handler) # type:
ignore[arg-type]
app.add_middleware(SlowAPIMiddleware)

```

`app.state.limiter` : slowapi s'attend à trouver le limiteur à cet emplacement précis de l'état de l'application pour fonctionner. `add_exception_handler` : quand une route dépasse son quota (`RateLimitExceeded` levée), ce handler intercepte l'exception et produit une réponse HTTP 429 propre plutôt qu'une erreur 500 brute. Le commentaire `type: ignore[arg-type]` désactive un avertissement du vérificateur de types `mypy` : la signature de `_rate_limit_exceeded_handler` est plus précise (spécifique à `RateLimitExceeded`) que celle générique attendue par Starlette (`Request`, `Exception`) — comportement correct à l'exécution, simple désaccord de typage entre les deux bibliothèques. `add_middleware(SlowAPIMiddleware)` active le comptage réel des requêtes.

```

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.cors_origins_list,
    allow_credentials=False,
    allow_methods=["GET", "POST"],
    allow_headers=["Content-Type"],
)

```

- `allow_origins=settings.cors_origins_list` : seules les origines listées (2.1) peuvent effectuer des requêtes cross-origin depuis un navigateur ;
- `allow_credentials=False` : aucun cookie ni en-tête d'authentification n'est partagé entre origines — cohérent avec une API sans état (pas de session) ;
- `allow_methods=["GET", "POST"]` : seules ces deux méthodes HTTP sont autorisées en cross-origin, reflétant exactement les méthodes réellement utilisées par les deux endpoints ;
- `allow_headers=["Content-Type"]` : seul cet en-tête personnalisé est autorisé dans les requêtes entrantes (nécessaire pour envoyer du JSON).

```

app.add_middleware(SecurityHeadersMiddleware)

app.include_router(router, prefix="/api/v1")
return app

app = create_app()

```

`add_middleware(SecurityHeadersMiddleware)` : ajouté **en dernier**, ce qui est important — Starlette empile les middlewares, et le dernier ajouté devient le plus « extérieur », donc le premier à voir la requête entrante et le dernier à voir la réponse sortante. Ça garantit que les en-têtes de sécurité sont posés sur *toute* réponse, y compris une erreur 429 de rate limit générée plus « à l'intérieur » de la pile.

`app.include_router(router, prefix="/api/v1")` : monte les routes définies dans `routes.py` (2.5) sous le préfixe `/api/v1` — `/health` devient donc accessible à `/api/v1/health`.

`app = create_app()` : exécuté une fois à l'import du module. C'est cet objet précis que `uvicorn` exécute au démarrage, référencé comme `app.main:app` dans le `Dockerfile` (2.10).

2.7 rules/anssi.py & anssi_rules.json

[backend/app/rules/](#) – scoring de conformité ANSSI (logique pure)

Module sans dépendance réseau ni base de données : calcul pur, testable unitairement, dupliqué en TypeScript côté frontend pour que le mot de passe n'ait jamais à quitter le navigateur pour être évalué.

```
_RULES_PATH = Path(__file__).parent / "anssi_rules.json"
```

`Path(__file__)` : chemin absolu du fichier `anssi.py` lui-même. `.parent` : le dossier qui le contient (`app/rules/`). L'opérateur `/` sur un objet `Path` concatène un segment de chemin de façon portable entre systèmes d'exploitation. Résultat : le chemin absolu vers `anssi_rules.json`, quel que soit le répertoire depuis lequel le processus Python a été lancé.

```
class AnssiSeverity(str, Enum):
    OK = "ok"
    WEAK = "weak"
    CRITICAL = "critical"
```

Hériter à la fois de `str` et `Enum` crée une énumération dont les membres sont *aussi* des chaînes de caractères utilisables directement (comparaison, sérialisation JSON) sans conversion explicite. Trois valeurs possibles, dans l'ordre croissant de gravité : conforme, faible, critique.

```
@dataclass(frozen=True)
class AnssiRuleResult:
    code: str
    label: str
    passed: bool
```

Résultat d'une règle individuelle : son identifiant technique (`code`, ex. `"min_length"`), son libellé lisible en français (`label`, ex. `"Au moins 12 caractères"`), et si elle est respectée (`passed`).

```
@dataclass(frozen=True)
class AnssiAnalysis:
    score: int # 0 à 100
    severity: AnssiSeverity
    rules: list[AnssiRuleResult]

    @property
    def is_compliant(self) -> bool:
        return all(r.passed for r in self.rules)
```

Résultat global de l'analyse : un score numérique, une sévérité, et le détail de chaque règle individuelle. `is_compliant` est un attribut calculé (pas stocké) : `all(...)` retourne `True` uniquement si **toutes** les règles de la liste ont `passed=True`, sans exception — un mot de passe peut avoir un score élevé sans être `compliant` si une seule règle échoue.

```
def _min_length(pwd: str, rule: dict[str, Any]) -> bool:
    return len(pwd) >= rule["value"]
```

Signature commune à toutes les fonctions de vérification : le mot de passe, et le dictionnaire JSON de la règle correspondante (d'où proviennent d'éventuels paramètres, ici `rule["value"] = 12`). Compare la longueur du mot de passe au minimum requis.

```
def _has_lower(pwd: str, _rule: dict[str, Any]) -> bool:
    return any(c.islower() for c in pwd)

def _has_upper(pwd: str, _rule: dict[str, Any]) -> bool:
    return any(c.isupper() for c in pwd)

def _has_digit(pwd: str, _rule: dict[str, Any]) -> bool:
    return any(c.isdigit() for c in pwd)
```

Trois fonctions structurellement identiques : `any(...)` retourne `True` dès qu'**au moins un** caractère du mot de passe satisfait la condition (minuscule / majuscule / chiffre). Le paramètre `_rule` est préfixé d'un `_` car il n'est pas utilisé dans le corps de ces trois fonctions — convention signalant un paramètre intentionnellement ignoré, requis uniquement pour respecter la signature commune de `_CHECKS`.

```
def _has_any_char(pwd: str, rule: dict[str, Any]) -> bool:
    return any(c in rule["chars"] for c in pwd)
```

Généralisation : vérifie qu'au moins un caractère du mot de passe appartient à l'ensemble `rule["chars"]` défini dans le JSON — utilisée pour la règle des caractères spéciaux (`!"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~`).

```
def _no_sequence(pwd: str, rule: dict[str, Any]) -> bool:
    lowered = pwd.lower()
    window = rule["window"]
    for pattern in rule["patterns"]:
        for i in range(len(pattern) - window + 1):
            if pattern[i : i + window] in lowered:
                return False
    return True
```

- `lowered = pwd.lower()` : la détection ignore la casse (« Azerty » est détecté comme « azerty ») ;
- `window = rule["window"]` : taille de la fenêtre glissante, 4 d'après le JSON ;
- boucle externe `for pattern in rule["patterns"]` : parcourt chaque motif de référence (`"0123456789"`, `"azertyuiop"` ...) ;
- boucle interne `for i in range(len(pattern) - window + 1)` : fait glisser une fenêtre de 4 caractères sur ce motif — pour `"azertyuiop"`, ça génère successivement `"azer"`, `"zert"`, `"erty"`, etc. ;
- `if pattern[i : i + window] in lowered` : si l'une de ces sous-chaînes de 4 caractères apparaît telle quelle dans le mot de passe, la règle échoue immédiatement (`return False`) ;

- si aucune fenêtre d'aucun motif n'a été trouvée après avoir tout parcouru, la règle est respectée (`return True`).

```
def _no_repeated_char(pwd: str, rule: dict[str, Any]) -> bool:
    max_run = rule["max_run"]
    run = 1
    for i in range(1, len(pwd)):
        run = run + 1 if pwd[i] == pwd[i - 1] else 1
        if run > max_run:
            return False
    return True
```

- `run = 1` : longueur de la séquence de caractères identiques consécutifs en cours, initialisée à 1 (le premier caractère compte pour lui-même) ;
- `for i in range(1, len(pwd))` : parcourt le mot de passe à partir du deuxième caractère (index 1), pour toujours pouvoir comparer à `pwd[i-1]` ;
- `run = run + 1 if pwd[i] == pwd[i - 1] else 1` : opérateur ternaire — si le caractère courant est identique au précédent, la séquence s'allonge (`run + 1`) ; sinon elle repart de 1 ;
- `if run > max_run: return False` : dès que la séquence dépasse la limite (3, d'après le JSON), la règle échoue immédiatement, sans attendre la fin du mot de passe ;
- si la boucle se termine sans jamais dépasser `max_run`, la règle est respectée.

```
_CHECKS: dict[str, Callable[[str, dict[str, Any]], bool]] = {
    "min_length": _min_length,
    "has_lower": _has_lower,
    "has_upper": _has_upper,
    "has_digit": _has_digit,
    "has_any_char": _has_any_char,
    "no_sequence": _no_sequence,
    "no_repeated_char": _no_repeated_char,
}
```

Dictionnaire de *dispatch* : associe la chaîne `"type"` de chaque règle du JSON à la fonction Python qui sait l'évaluer. L'annotation `Callable[[str, dict[str, Any]], bool]` documente précisément la signature attendue pour chaque valeur du dictionnaire : une fonction prenant une chaîne et un dictionnaire, retournant un booléen. Ce niveau d'indirection permet d'ajouter une règle dans le JSON en réutilisant un `type` déjà supporté, sans toucher au code Python.

```
class AnssiComplianceChecker:
    """Évalue un mot de passe selon les règles définies dans anssi_rules.json."""

    def __init__(self) -> None:
        self._spec = json.loads(_RULES_PATH.read_text(encoding="utf-8"))
```

`_RULES_PATH.read_text(encoding="utf-8")` lit tout le contenu du fichier JSON sous forme de chaîne de caractères. `json.loads(...)` le convertit en structure Python native (dictionnaires et listes imbriqués). Cette lecture se fait **une seule fois**, à la création de l'instance `AnssiComplianceChecker`, pas à chaque appel d'`analyze`.

```
def analyze(self, password: str) -> AnssiAnalysis:
    rules = self._spec["rules"]
```

Extrait la liste des règles depuis la spécification JSON chargée en mémoire.

```
results = [
    AnssiRuleResult(r["code"], r["label"], _CHECKS[r["type"]](password, r))
    for r in rules
]
```

Pour chaque règle `r` du JSON : `_CHECKS[r["type"]]` récupère la fonction de vérification correspondante dans le dictionnaire de dispatch, puis l'appelle immédiatement avec `(password, r)` — le résultat booléen devient le troisième argument du constructeur `AnssiRuleResult(code, label, passed)`. Une seule ligne condense donc : sélection de la fonction, exécution, et construction de l'objet résultat.

```
passed_count = sum(1 for r in results if r.passed)
score = round(100 * passed_count / len(rules))
```

`sum(1 for r in results if r.passed)` : compte le nombre de règles réussies (une expression génératrice qui produit `1` pour chaque résultat où `passed` est vrai, puis les additionne). Le score est le pourcentage arrondi de règles réussies — ex. 6 règles sur 7 : `round(100 * 6 / 7) = 86`.

```
thresholds = self._spec["severity_thresholds"]
if score >= thresholds["ok_score"]:
    severity = AnssiSeverity.OK
elif score >= thresholds["weak_min_score"]:
    severity = AnssiSeverity.WEAK
else:
    severity = AnssiSeverity.CRITICAL
```

Applique les seuils définis dans le JSON (`ok_score: 100`, `weak_min_score: 60`) : il faut un score de **100** (donc toutes les règles, sans exception) pour être classé `OK` ; entre 60 et 99 inclus, `WEAK` ; en dessous de 60, `CRITICAL`. L'ordre des comparaisons (du seuil le plus haut au plus bas, avec `elif`) est important : un score de 100 satisferait aussi la condition `>= 60`, mais elle n'est jamais atteinte car la première condition, plus stricte, l'a déjà capté.

```
return AnssiAnalysis(score=score, severity=severity, rules=results)
```

Assemble et retourne l'objet de résultat final, immuable, portant le score, la sévérité et le détail de chaque règle individuelle — c'est cet objet que consomment les tests (2.13).

anssi_rules.json — les règles en données

CODE	RÈGLE	PARAMÈTRE
<code>min_length</code>	Au moins 12 caractères	<code>value: 12</code>
<code>lowercase</code>	Contient une minuscule	—
<code>uppercase</code>	Contient une majuscule	—
<code>digit</code>	Contient un chiffre	—
<code>special</code>	Contient un caractère spécial	<code>chars: !"#\$%&'()*+,-./:;<=>?@[\\]^_`{ }~</code>
<code>no_sequence</code>	Pas de suite évidente	<code>patterns: [0123456789, abc...z, azerty..., qwerty...], window: 4</code>
<code>no_repeat</code>	Pas 4 répétitions ou plus	<code>max_run: 3</code>

Seuils de sévérité : `ok_score: 100`, `weak_min_score: 60`. Ce fichier n'est pas codé en dur dans `anssi.py` précisément pour pouvoir être partagé tel quel avec le frontend (copie dans `frontend/src/anssi-rules.json`) : un test dédié (2.14) garantit que les deux copies ne divergent jamais silencieusement.

2.8 database/init.sql

[database/init.sql](#) – schéma PostgreSQL

```
CREATE UNLOGGED TABLE IF NOT EXISTS breached_passwords (
    hash_prefix    CHAR(5)      NOT NULL,
    hash_suffix    VARCHAR(35)  NOT NULL,
    occurrence_count INTEGER    NOT NULL DEFAULT 1,
    PRIMARY KEY (hash_prefix, hash_suffix)
);
```

- `UNLOGGED` : PostgreSQL n'écrit pas les modifications de cette table dans le WAL (write-ahead log), ce qui accélère fortement les imports en masse — en contrepartie, la table est vidée automatiquement après un arrêt brutal du serveur (crash), pas après un arrêt propre. Acceptable ici car ce sont des données publiques entièrement reconstituables en relançant l'importeur ;

- `hash_prefix CHAR(5)` : type de longueur fixe (toujours 5 caractères), plus efficace en stockage que `VARCHAR` quand la longueur est garantie constante ;
- `hash_suffix VARCHAR(35)` : 40 caractères du SHA-1 hexadécimal moins les 5 du préfixe = 35 ;
- `occurrence_count INTEGER NOT NULL DEFAULT 1` : nombre de fois que ce mot de passe précis apparaît dans le corpus RockYou ;
- `PRIMARY KEY (hash_prefix, hash_suffix)` : clé primaire **composite** — c'est elle qui sert d'index B-Tree pour les recherches `WHERE hash_prefix = $1` de `db.py`, `hash_prefix` étant la colonne de tête de l'index.

2.9 .env.example

[backend/.env.example](#) – modèle de configuration locale

```
DATABASE_URL=postgresql://rockyou:rockyou@postgres:5432/rockyou
API_CORS_ORIGINS=http://localhost:5173
RATE_LIMIT_PER_MINUTE=30
```

Fichier **modèle**, destiné à être copié en `.env` (non versionné) et adapté localement. Chaque ligne correspond exactement à un attribut de la classe `Settings` vue en 2.1, en majuscules (convention des variables d'environnement). C'est `pydantic-settings` qui fait le lien automatique entre `DATABASE_URL` et l'attribut `database_url`.

2.10 Dockerfile

[backend/Dockerfile](#) – image de production

```
FROM python:3.12-slim AS base
```

Image de base officielle Python 3.12, variante `slim` : contient l'essentiel pour exécuter Python sans les outils de compilation ni la documentation présents dans l'image complète — empreinte disque réduite.

```
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PIP_NO_CACHE_DIR=1
```

`PYTHONDONTWRITEBYTECODE=1` : empêche Python d'écrire des fichiers `.pyc` sur disque (inutiles dans un conteneur éphémère). `PYTHONUNBUFFERED=1` : force l'affichage immédiat des logs (`print`, `logging`) sans mise en tampon — essentiel pour voir les logs en temps réel via `docker logs`. `PIP_NO_CACHE_DIR=1` : `pip` ne conserve pas de cache de téléchargement, réduisant la taille de l'image finale.

```
WORKDIR /app
```

Définit `/app` comme répertoire de travail pour toutes les instructions suivantes (`COPY`, `RUN`, et la commande finale).

```
RUN addgroup --system app && adduser --system --ingroup app app
```

Crée un groupe système et un utilisateur système, tous deux nommés `app`, sans mot de passe ni shell interactif (`--system`). Préparation pour ne **jamais** exécuter l'application en tant que `root` à l'intérieur du conteneur — bonne pratique de sécurité standard.

```
COPY requirements.txt .  
RUN pip install --no-cache-dir -r requirements.txt
```

Copie **uniquement** `requirements.txt` avant le reste du code source, puis installe les dépendances. Cet ordre exploite le cache de build Docker : tant que `requirements.txt` ne change pas, cette étape coûteuse (téléchargement de paquets) n'est pas ré-exécutée à chaque modification du code applicatif.

```
COPY app ./app
```

Copie le code source de l'application **après** l'installation des dépendances — c'est l'étape la plus susceptible de changer à chaque build, donc placée en dernier pour maximiser la réutilisation du cache des étapes précédentes.

```
USER app
```

Bascule l'utilisateur d'exécution de `root` vers l'utilisateur non-privilégié créé plus haut, pour toutes les instructions suivantes et pour le processus final du conteneur.

```
EXPOSE 8000
```

Déclaration documentaire indiquant que le conteneur écoute sur le port 8000 — n'ouvre pas réellement le port (ça se fait via `docker run -p` ou `docker-compose.yml`), sert d'information pour les outils qui inspectent l'image.

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=10s \  
  CMD python -c "import urllib.request;  
  urllib.request.urlopen('http://localhost:8000/api/v1/health')" || exit 1
```

Docker interroge l'endpoint `/api/v1/health` (vu en 2.5) toutes les 30 secondes, avec un délai de grâce de 10 secondes après le démarrage du conteneur (le temps que l'application s'initialise) et un timeout de 3 secondes par tentative. Si la requête échoue, le conteneur est marqué `unhealthy`, visible par `docker ps` et exploitable par un orchestrateur pour redémarrer le service automatiquement.

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Commande exécutée au démarrage du conteneur : lance `uvicorn` (le serveur ASGI, dépendance `requirements.txt`) en pointant vers l'objet `app` défini dans `app/main.py` (2.6). `--host 0.0.0.0` : écoute sur toutes les interfaces réseau du conteneur (nécessaire pour être joignable depuis l'extérieur du conteneur, `127.0.0.1` ne suffirait pas). `--port 8000` : correspond au port déclaré par `EXPOSE`.

2.11 pyproject.toml

[backend/pyproject.toml](#) – configuration des outils

```
[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B"]
```

`ruff` est un linter/formateur Python (remplace généralement `flake8` + `isort` + `pyupgrade`, en beaucoup plus rapide). `line-length = 100` : longueur de ligne maximale tolérée. `target-version = "py312"` : `ruff` adapte ses règles de modernisation à la syntaxe disponible en Python 3.12. `select` active des familles de règles précises : `E` (erreurs de style PEP 8), `F` (erreurs Pyflakes — imports inutilisés, variables non définies...), `I` (tri des imports), `UP` (suggestions de modernisation de syntaxe), `B` (bugbear — détection de pièges Python courants).

```
[tool.mypy]
python_version = "3.12"
ignore_missing_imports = true
```

`mypy` est un vérificateur de types statique. `ignore_missing_imports = true` : ne signale pas d'erreur pour les bibliothèques tierces qui n'exposent pas d'annotations de type (« stubs ») — évite un bruit de faux positifs sur des dépendances non maîtrisées par le projet.

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
asyncio_default_fixture_loop_scope = "function"
```

`testpaths = ["tests"]` : `pytest` ne cherche des tests que dans le dossier `tests/`. `pythonpath = ["."]` : ajoute la racine du projet au chemin d'import Python, pour que `from app.rules.anssi import ...` fonctionne depuis les fichiers de test sans installation préalable du package. `asyncio_default_fixture_loop_scope = "function"` : réglage de `pytest-asyncio` — chaque test asynchrone reçoit sa propre boucle événementielle, isolée des autres tests.

2.12 requirements-dev.txt

[backend/requirements-dev.txt](#) – dépendances de développement

```
-r requirements.txt
pytest==8.3.3
pytest-asyncio==0.24.0
httpx==0.27.2
ruff==0.6.9
mypy==1.11.2
```

- `-r requirements.txt` : inclut d'abord toutes les dépendances de production (2.10/section 4) — les outils de dev s'ajoutent par-dessus, ils ne les remplacent pas ;
- `pytest` : le framework de test utilisé par les deux fichiers de tests (2.13, 2.14) ;
- `pytest-asyncio` : extension permettant à `pytest` d'exécuter des fonctions de test `async def` — nécessaire dès qu'on veut tester du code appelant `asyncpg` ou des endpoints FastAPI asynchrones ;
- `httpx` : client HTTP utilisable de façon asynchrone, typiquement pour piloter un `TestClient` FastAPI dans de futurs tests d'intégration des endpoints ;
- `ruff` / `mypy` : les mêmes outils configurés dans `pyproject.toml` (2.11), présents ici comme dépendances installables.

Ce fichier n'est jamais installé dans l'image Docker de production (2.10 n'installe que `requirements.txt`) — il n'est utilisé qu'en local ou en intégration continue.

2.13 tests/test_anssi.py

[backend/tests/test_anssi.py](#) – tests du scoring ANSSI

```
from app.rules.anssi import AnssiComplianceChecker, AnssiSeverity

checker = AnssiComplianceChecker()
```

Une seule instance de `AnssiComplianceChecker` (2.7) créée au niveau du module de test, réutilisée par toutes les fonctions de test ci-dessous — le JSON des règles n'est donc lu qu'une fois pour toute la suite de tests.

```
def test_weak_password_is_critical():
    result = checker.analyze("azerty")
    assert result.severity == AnssiSeverity.CRITICAL
    assert result.is_compliant is False
```

Un mot de passe trivial (trop court, pas de majuscule, pas de chiffre, motif clavier connu) doit échouer la quasi-totalité des règles et être classé `CRITICAL`, et par construction non conforme.

```
def test_strong_password_is_compliant():
    result = checker.analyze("K7#mPzq2Xrw!")
    assert result.is_compliant is True
    assert result.severity == AnssiSeverity.OK
    assert result.score == 100
```

Vérifie le cas nominal inverse : un mot de passe de 12 caractères mélangeant majuscules, minuscules, chiffres et symboles, sans suite ni répétition, doit obtenir le score maximal et être conforme.

```
def test_sequential_pattern_detected():
    result = checker.analyze("Abcdefgh1234!")
    rule = next(r for r in result.rules if r.code == "no_sequence")
    assert rule.passed is False
```

`next(r for r in result.rules if r.code == "no_sequence")` : extrait **précisément** le résultat de la règle `no_sequence` parmi toutes celles retournées, pour tester son verdict indépendamment du score global. `"Abcdefgh1234!"` contient à la fois `"abcd"` (issu du motif alphabétique) et `"1234"` — la règle doit donc échouer.

```
def test_repeated_char_detected():
    result = checker.analyze("Aaaaaaaa1234!")
    rule = next(r for r in result.rules if r.code == "no_repeat")
    assert rule.passed is False
```

Sept `"a"` consécutifs dépassent largement la limite de 3 répétitions autorisées — la règle `no_repeat` doit échouer.

```
def test_three_repeated_chars_is_allowed():
    result = checker.analyze("Aaa1#zzzB234!")
    rule = next(r for r in result.rules if r.code == "no_repeat")
    assert rule.passed is True
```

Teste la **limite exacte** : trois répétitions consécutives (`"zzz"`) sont explicitement autorisées (`max_run: 3` dans le JSON, la règle échoue seulement à partir de 4). C'est un test de cas limite (« boundary test »), essentiel pour vérifier qu'une inégalité stricte n'a pas été inversée par erreur dans `_no_repeated_char`.

```
def test_min_length_boundary():
    just_short = checker.analyze("Ab3#Ab3#Ab3") # 11 caractères
    exactly_min = checker.analyze("Ab3#Ab3#Ab34") # 12 caractères
    assert next(r for r in just_short.rules if r.code == "min_length").passed is False
    assert next(r for r in exactly_min.rules if r.code == "min_length").passed is True
```

Autre test de cas limite, sur la longueur minimale cette fois : 11 caractères doivent échouer, exactement 12 doivent réussir — vérifie que la comparaison dans `_min_length` est bien `>=` et non `>`.

2.14 tests/test_ansi_rules_sync.py

[backend/tests/test_ansi_rules_sync.py](#) – cohérence backend / frontend

```
_BACKEND_RULES = Path(__file__).parent.parent / "app" / "rules" / "anssi_rules.json"
_FRONTEND_RULES = Path(__file__).parent.parent.parent / "frontend" / "src" / "anssi-
rules.json"
```

Construit les deux chemins absolus par remontées successives de dossiers depuis l'emplacement du fichier de test : `parent.parent` remonte de `tests/` à la racine `backend/` pour atteindre `app/rules/anssi_rules.json` ; une remontée supplémentaire (`parent.parent.parent`) sort du dossier `backend/` pour atteindre le dossier frère `frontend/src/anssi-rules.json`, à la racine du dépôt.

```
def test_anssi_rules_identical_between_backend_and_frontend():
    backend_rules = json.loads(_BACKEND_RULES.read_text(encoding="utf-8"))
    frontend_rules = json.loads(_FRONTEND_RULES.read_text(encoding="utf-8"))
    assert backend_rules == frontend_rules
```

Charge les deux fichiers JSON et compare les structures Python résultantes avec `==` — une comparaison profonde et récursive (dictionnaires, listes, valeurs scalaires), indépendante du formatage exact du fichier (espaces, ordre d'écriture des clés au même niveau important toutefois pour les dictionnaires Python... en réalité `==` sur des dicts ignore l'ordre des clés). Ce test transforme une divergence silencieuse entre les deux copies des règles ANSSI — qui casserait la promesse « même analyse des deux côtés » sans qu'aucune erreur ne soit visible autrement — en échec de test explicite et immédiat dès qu'une des deux copies est modifiée sans l'autre.

3. Bibliothèques utilisées

Contenu de `backend/requirements.txt`, avec le rôle de chaque dépendance dans ce projet et sa licence.

BIBLIOTHÈQUE	VERSION ÉPINGLÉE	RÔLE DANS CE PROJET	LICENCE
<code>fastapi</code>	<code>0.115.0</code>	Framework web ASGI : définition des routes, validation automatique via Pydantic, génération de la documentation OpenAPI (<code>/api/docs</code>). Colonne vertébrale du backend.	MIT
<code>uvicorn[standard]</code>	<code>0.30.6</code>	Serveur ASGI qui exécute réellement l'application (<code>uvicorn app.main:app</code> dans le <code>Dockerfile</code>). L'extra <code>[standard]</code> ajoute des dépendances de performance (ex. boucle événementielle <code>uvloop</code>).	BSD-3-Clause
<code>asyncpg</code>	<code>0.29.0</code>	Driver PostgreSQL asynchrone, utilisé dans <code>db/db.py</code> pour le pool de connexions et la requête de recherche par préfixe. Choisi pour sa vitesse (pas d'ORM, requêtes SQL directes).	Apache License 2.0
<code>pydantic</code>	<code>2.9.2</code>	Validation de données et sérialisation. Base des modèles de <code>api/schemas.py</code> et de la configuration dans <code>core/config.py</code> .	MIT
<code>pydantic-settings</code>	<code>2.5.2</code>	Extension de Pydantic pour charger la configuration depuis les variables d'environnement / <code>.env</code> (<code>core/config.py</code>). Séparé de <code>pydantic</code> depuis Pydantic v2.	MIT
<code>slowapi</code>	<code>0.1.9</code>	Limitation de débit (rate limiting) par IP, pour empêcher l'énumération massive de préfixes de hash sur <code>/breach-check</code> (<code>core/rate_limit.py</code> , <code>main.py</code> , <code>api/routes.py</code>).	MIT
<code>python-dotenv</code>	<code>1.0.1</code>	Lecture du fichier <code>.env</code> pour le développement local. Utilisé en interne par <code>pydantic-settings</code> .	BSD-3-Clause

Dépendances de développement (`requirements-dev.txt`, section 2.12)

BIBLIOTHÈQUE	VERSION	RÔLE	LICENCE
<code>pytest</code>	<code>8.3.3</code>	Framework d'exécution des tests	MIT
<code>pytest-asyncio</code>	<code>0.24.0</code>	Support des tests <code>async def</code>	Apache License 2.0
<code>httpx</code>	<code>0.27.2</code>	Client HTTP pour de futurs tests d'intégration des endpoints	BSD-3-Clause
<code>ruff</code>	<code>0.6.9</code>	Linter / formatteur	MIT
<code>mypy</code>	<code>1.11.2</code>	Vérification statique de types	MIT

Sur la fraîcheur de ces versions : toutes ces bibliothèques appartiennent à une même génération de l'écosystème FastAPI/Pydantic (autour de Pydantic 2.9, publiée fin 2024). Je ne peux pas garantir qu'il s'agit des toutes dernières versions disponibles aujourd'hui sans une vérification réseau, qui n'a pas pu être menée à bien dans cette session. La seule façon fiable de confirmer : `pip list --outdated` et `pip-audit -r requirements.txt` (donné en section 0).

Sur les licences : MIT, BSD-3-Clause et Apache-2.0 sont toutes des licences open-source permissives : elles autorisent l'usage commercial, la modification et la redistribution, avec pour seule obligation réelle de conserver la mention de copyright/licence d'origine. Aucune n'impose de rendre votre propre code open-source (contrairement à une licence copyleft comme la GPL). Aucune incompatibilité entre elles.